

RAGE: Robust Agentic Security Gateway for Text-to-SQL — Defensa Contra Ataques Crescendo Multi-Turno

Autores: Equipo de Investigación RAGE

Evento: Global South AI Safety Hackathon, junio de 2026

Repositorio: rage-multiturn — Python 3.12, licencia MIT

Ancla arXiv: Russinovich et al., 2404.01833 [cs.CR]

Resumen

Las interfaces Text-to-SQL que conectan modelos de lenguaje a bases de datos relacionales exponen una superficie de ataque donde el adversario puede migrar gradualmente una sesión legítima hacia consultas destructivas sin activar filtros stateless. Russinovich et al. demostraron que Crescendo alcanza tasas de éxito del 98-100% en modelos alineados mediante prompts exclusivamente benignos distribuidos en N turnos. Presentamos RAGE (Robust Agentic Security Gateway for Text-to-SQL), un framework de cuatro capas con filtro semántico stateful, motor de decisión EWMA con trinquete de advertencias y gateway SQL endurecido. Integramos el Training-Center, entorno interactivo de simulación y calibración de hiperparámetros. Introducimos las métricas AUC-D (Área Bajo la Curva de Degradación) y TRI (Temporal Resistance Index). Validamos el sistema con 108 pruebas automatizadas (100% aprobadas) y el escenario SCENARIO_CRESCENDO, donde RAGE bloquea los turnos T4 y T5 mediante acción independiente de capas 3, 4 y gateway.

1. Introducción y Marco Teórico

1.1 La Superficie de Ataque en Text-to-SQL

Las organizaciones despliegan cada vez más agentes conversacionales capaces de traducir lenguaje natural a SQL sobre datos operacionales [2, 3]. A diferencia de formularios web o APIs REST con esquemas de validación rígidos, estos sistemas aceptan entradas de longitud arbitraria cuya intención semántica es difícil de acotar con firmas estáticas. Un usuario que mantiene una conversación multi-turno dispone de una libertad expresiva incompatible con el principio de mínimo privilegio en seguridad de bases de datos.

Las defensas previas evalúan cada turno de forma aislada: escanean el prompt actual en busca de patrones conocidos (DROP TABLE, IGNORE PREVIOUS INSTRUCTIONS) y lo rechazan si alguna firma coincide. Este modelo stateless es un corolario directo de los benchmarks de alineación mono-turno que dominan la investigación actual. Russinovich et al. lo formulan explícitamente: "all current benchmarks focus solely on single-turn jailbreaks" [1, §1], y demuestran que Crescendo evade estas medidas con facilidad.

1.2 Teoría del Cambio: De Filtros Apátridas a Defensas Basadas en Estado

La contribución conceptual central de RAGE es la Teoría del Cambio hacia defensas session-aware. El

adversario persistente no busca activar una única alarma; construye momentum conversacional que coloniza el contexto del modelo turno a turno. Neutralizar este adversario exige tres capacidades ausentes en filtros stateless:

1. Anclaje a línea base: medir desviación acumulada respecto al turno T_0 , no solo respecto al turno inmediatamente anterior.
2. Memoria de riesgo de sesión: acumular señales moderadas que, individualmente, no superan umbrales per-turno.
3. Contención determinista de acciones: validar la consulta SQL resultante independientemente de lo que las capas semánticas permitan.

Formalmente, sea $S = \{ q_0, q_1, \dots, q_N \}$ una secuencia de prompts y $e_i \in \mathbb{R}^{2048}$ el embedding L2-normalizado del turno i . Definimos la deriva turno-a-turno $\delta_i = \max(0, 1 - e_i \cdot e_{i-1})$ y la deriva acumulada $\Delta_i = \max(0, 1 - e_i \cdot e_0)$. Un filtro stateless con umbral τ bloquea el turno i iff $\delta_i > \tau$. En un ataque Crescendo con tamaño de paso $\epsilon \ll \tau$, todos los turnos satisfacen $\delta_i = \epsilon < \tau$, pero $\Delta_N = \sum_{i=1}^N \epsilon \approx N\epsilon$ puede superar ampliamente τ para N suficientemente grande.

Proposición 1 (Ceguera del Filtro Apátrida). Un filtro que evalúa exclusivamente δ_i es ciego ante trayectorias de escalada gradual cuyo vector semántico converge monótonamente lejos de e_0 sin saltos abruptos entre turnos consecutivos. La defensa requiere computar Δ_i y mantener un score de riesgo de sesión R_t con memoria exponencial.

2. Trabajos Relacionados y Análisis de la Amenaza Crescendo

2.1 Literatura Previa

Crescendo (Microsoft, 2024). Russinovich, Salem y Eldan [1] introducen un jailbreak multi-turno de caja negra que usa exclusivamente prompts benignos y legibles. Su mecanismo se fundamenta en el principio psicológico foot-in-the-door: acordar una petición pequeña incrementa sistemáticamente la compliance con demandas mayores. En LLaMA-2 70b, la secuencia completa $A \rightarrow B \rightarrow C$ alcanza 99,9% de éxito, mientras que B aislado logra 36,2% y C solo 17,3% — confirmando que es la trayectoria, no el turno individual, la que constituye el ataque.

Fallas de alineación y ataques universales. Zou et al. [5] demuestran ataques adversarios transferibles sobre modelos alineados; Pérez y Ribeiro [6] documentan técnicas de Ignore Previous Prompt. Estos trabajos operan predominantemente en el régimen mono-turno.

OWASP LLM Top 10. La taxonomía OWASP [4] identifica vectores directamente relevantes: LLM01 (Prompt Injection), LLM06 (Excessive Agency — agencia excesiva del agente sobre herramientas), y LLM08 (confianza indebida en entradas no confiables, incluidos resúmenes de intención generados por el propio atacante). RAGE mapea explícitamente cada capa a estos identificadores.

Crescendomation. Russinovich et al. reportan que la variante automatizada supera PAIR y MSJ entre 29-61% en GPT-4 [1, Tabla 4], elevando la urgencia de defensas session-aware desplegables.

2.2 Tres Fases del Ataque Adaptadas a Text-to-SQL

Fase | Turnos | Objetivo del adversario | Señal stateless

Context Seeding | T0-T1 | Establecer competencia técnica; consultas legítimas sobre sales | Ninguna

Scope Expansion | T2-T3 | Ampliar tablas (products), JOINS, marco de "auditoría" | δ_i bajo; Δ_i crece lentamente

Payload Injection | T4+ | UNION ALL SELECT ... FROM system_config; exfiltración | Payload enmascarado como continuación natural

El bypass confirmado pre-corrección en el gateway — el patrón `\bUNION\s+SELECT\b` no coincidía con `UNION ALL SELECT` — permitía que una rama `UNION` accediera a tablas no autorizadas mientras el extractor `_FROM_TABLE_RE` validaba solo la primera cláusula `FROM`.

3. Metodología Detallada y Arquitectura de Software

RAGE implementa una cascada de cuatro capas seguida de un gateway de acciones y un evaluador de métricas temporales:

```
Turno del usuario ? [L1: Regex] ? [L2: RAG KB] ? [L3: Filtro Semántico] ? [L4: Motor de Decisión]
                                                                ?
                                                                [Gateway SQL] ? [Agente SQLite]
                                                                ?
                                                                [Evaluador AUC-D / TRI]
```

3.1 Capa 1 — Pre-Filtro Determinístico (layer1_rules.py)

La Capa 1 aplica 14 reglas regex compiladas (L1-001 a L1-014) con salida temprana en la primera coincidencia. Coste: $O(1)$ por turno, sin ML ni llamadas API.

```
_RAW_RULES: list[tuple[str, str, str]] = [
    ("L1-001", "Explicit ignore-previous-instructions", r"ignore\s+(all\s+)?(previous|prior|above)\s+instructi",
    ("L1-006", "SQL DROP TABLE attempt", r"\bDROP\s+TABLE\b"),
    ("L1-007", "SQL GRANT PRIVILEGES", r"\bGRANT\s+ALL\s+PRIVILEGES\b"),
    # ... L1-002 a L1-014: DAN, shell exec, exfiltración, prompt leakage ...
]

class DeterministicPreFilter:
    def evaluate(self, text: str) -> Layer1Signal:
        for rule in _COMPILED_RULES:
            m = rule.pattern.search(text)
            if m:
                return Layer1Signal(matched=True, pattern_id=rule.rule_id, matched_text=m.group(0))
        return Layer1Signal(matched=False)
```

Contribución al score: +70 puntos ante coincidencia determinística. Limitación reconocida: invisible ante atacantes Crescendo que evitan firmas conocidas; diseñada como trip-wire rápido, no como defensa primaria multi-turno.

3.2 Capa 2 — Base de Conocimiento RAG (layer2_rag.py)

La Capa 2 embebe el texto del turno y calcula similitud coseno contra una base vectorial local de 34 ejemplos de ataque OWASP (familias LLM01, inyección indirecta, payload splitting, escalada gradual, etc.) almacenados en `rage_core/kb/threats.json`.

```
class ThreatKBRetriever:
    def score(self, text: str) -> Layer2Signal:
        query_vec = self._embed_query(text)
        sims = _cosine_similarity(query_vec, self._matrix)
        top_idx = int(np.argmax(sims))
        top_sim = float(sims[top_idx])
        if top_sim < self._threshold:
            return Layer2Signal(score=top_sim)
        threat = self._threats[top_idx]
        return Layer2Signal(score=top_sim, top_match_id=threat["id"], owasp_id=threat["owasp_id"], ...)
```

Prioridad de embedder: sentence-transformers → OpenAI → TF-IDF offline (default). La función `add_threat()` permite hot-update en runtime sin reentrenamiento. Contribución al score: hasta +30 puntos ($\min(L2_score, 1) \times 30$).

3.3 Capa 3 — Filtro Semántico Stateful (layer3_semantic.py)

Núcleo de la defensa anti-Crescendo. Utiliza `HashingVectorizer` con `n_features=2048`, `alternate_sign=False`, `norm="l2"`, garantizando compatibilidad dimensional entre turnos sin refitting:

$$\cos(u, v) = \frac{u \cdot v}{\|u\| \|v\|} = 1$$

El bloque central con `np.dot()` implementa ambas métricas de deriva:

```
# 2. Turn-to-turn drift
if state.turn_embeddings:
    prev_emb = np.array(state.turn_embeddings[-1], dtype=np.float32)
    cos_sim = float(np.dot(current_emb, prev_emb))
    drift = max(0.0, 1.0 - cos_sim)

# 3. Cumulative drift: distance from conversation baseline (turn 0)
cumulative_drift = 0.0
if len(state.turn_embeddings) >= 1:
    baseline_emb = np.array(state.turn_embeddings[0], dtype=np.float32)
    baseline_cos_sim = float(np.dot(current_emb, baseline_emb))
    cumulative_drift = max(0.0, 1.0 - baseline_cos_sim)

suspicious = drift > self._threshold or cumulative_drift > self._threshold
```

Con umbral default $\tau = 0.80$ (calibrado para `HashingVectorizer`; ≈ 0.35 con `sentence-transformers`). Cumplimiento OWASP LLM08: `_sanitize()` elimina caracteres de control (`\x00-\x1f`, `\x7f`) y trunca resúmenes a 300 caracteres antes de almacenamiento o inclusión en prompts del juez LLM opcional (gpt-4o-mini).

3.4 Capa 4 — Motor de Decisión Dinámico (layer4_decision.py)

Fusión de scores per-turno

$$s_i = 70 \cdot 1[L1] + 30 \cdot \min(L2, 1) + 20 \cdot \min(\max(\delta_i, \Delta_i), 1) + 5 \cdot 1[\text{turn} \geq 2 \wedge \Delta_i > 0.75] + 10 \cdot 1[\text{LLM flagged}]$$

Bandas: $\theta_{\text{warn}} = 30$, $\theta_{\text{block}} = 65$.

EWMA de riesgo de sesión

$$R_t = (1 - \alpha) \cdot R_{t-1} + \alpha \cdot \frac{s_i}{100}, \quad \alpha = 0.50, \quad R_0 = 0$$

```

_EWMA_ALPHA: float = 0.5
_RATCHET_TURNS: int = 2
_SESSION_RISK_WARN_THRESHOLD: float = 0.18
_SESSION_RISK_BLOCK_THRESHOLD: float = 0.40

state.session_risk_score = (
    (1.0 - self.ewma_alpha) * state.session_risk_score
    + self.ewma_alpha * normalised_score
)

```

Tabla de escalada de riesgo (turnos moderados con $s_i = 20$):

Turno	s_i	$s_i/100$	$R_t (\alpha=0.5)$	Elevación
0	20	0.20	0.100	—
1	22	0.22	0.160	—
2	24	0.24	0.200	WARN ($R_t > 0.18$)
3	26	0.26	0.230	WARN
4	28	0.28	0.255	BLOCK (ratchet K=2)

Trinquete de advertencias consecutivas

Tras $K_{\text{ratchet}} = 2$ turnos consecutivos en banda WARN, la banda se eleva incondicionalmente a BLOCK. El contador consecutivo_warns se reinicia en ALLOW o BLOCK.

3.5 Gateway de Seguridad SQL (gateway.py)

Última línea de defensa determinista antes de ejecutar `query_db()`. Corrección del bypass UNION ALL:

```

("UNION-based exfiltration", re.compile(r"\bUNION\b", re.IGNORECASE)),

_ALL_TABLES_RE = re.compile(r"\b(?:FROM|JOIN)\s+(\w+)\b", re.IGNORECASE)

tables_found = _ALL_TABLES_RE.findall(sql)
for table in tables_found:
    if table.lower() not in _ALLOWED_TABLES:
        return False, f"Table '{table}' is not in the allowlist {_ALLOWED_TABLES}"

```

Tablas permitidas: sales, products, regions. Blocklist ampliada con 21 patrones: ALTER, CREATE, EXEC, SLEEP/BENCHMARK, CHAR(), literales hex, information_schema, sqlite_master, LOAD_FILE, etc.

4. Entorno de Simulación y Aprendizaje: Infraestructura Training-Center

El módulo `rage_core/training/` y el directorio `Training-Center/` constituyen la contribución de infraestructura interactiva desarrollada durante el hackatón de junio de 2026. Su propósito es triple: (i) simular escenarios de ataque/defensa reproducibles, (ii) calibrar hiperparámetros (`ewma_alpha`, `ratchet_turns`, umbrales de sesión), y (iii) generar candidatos para hot-update del KB ante nuevas variantes de Crescendomatation.

4.1 Arquitectura

Componente | Archivo | Función

Orquestador | orchestrator.py | Ejecuta un escenario turno a turno con/sin RAGE

Escenarios | escenarios.py | Puente entre demo/attacks.py y packs JSON custom

Campaña | campaign.py | Agrega ASR defended vs baseline

Reporter | reporter.py | Deriva insights y candidatos KB

Apply | apply.py | Aplica parches a threats.json

CLI | cli.py | uv run rage-training

El orquestador materializa cada turno como TurnRecord con trazabilidad completa:

```
class ScenarioOrchestrator:
    def run(self, pack: ScenarioPack, defended: bool, mode: str, iteration: int = 1) -> ScenarioRunResult:
        pipeline = DefensePipeline() if defended else None
        state = ConversationState()
        agent = SalesAgent(defended=defended)
        for i, turn in enumerate(pack.turns):
            if defended and pipeline:
                signal = pipeline.evaluate(turn.user_text, state)
                band = signal.band
                session_risk = state.session_risk_score
            # ... gateway check, ground-truth scoring, vulnerability tagging ...
```

La campaña ejecuta cada escenario con RAGE y sin RAGE (baseline), calculando reducción de ASR:

```
class TrainingCampaign:
    def run(self, packs: list[ScenarioPack] | None = None) -> CampaignResult:
        for pack in scenario_list:
            runs.append(self._orchestrator.run(pack, defended=True, mode="with_rage"))
            if self.include_baseline:
                runs.append(self._orchestrator.run(pack, defended=False, mode="baseline_no_rage"))
```

Los resultados se exportan a Training-Center/results/crescendo_YYYYMMDD_HHMMSS.json. El módulo build_actionable_insights() detecta bypasses de bajo score, bypasses en banda WARN, y genera entradas KB candidatas (tc-scenario_id-tturn) para aplicación via uv run rage-training-apply --apply-kb.

4.2 Calibración de Hiperparámetros

El flujo de calibración recomendado:

1. uv run rage-training --scenarios crescendo_escalation — medir ASR defended vs baseline.
2. Inspeccionar session_risk_score y consecutive_warns por turno en los JSON de resultados.
3. Ajustar _SESSION_RISK_WARN_THRESHOLD (default 0.18) si T3 no eleva a WARN.
4. Ajustar _RATCHET_TURNS (default 2) si el atacante acampa en WARN sin alcanzar BLOCK.
5. Re-ejecutar campaña y verificar que test_gradual_escalation_scenario sigue pasando.

Complementariamente, uv run rage-redteam ejecuta un loop adaptativo que intenta romper RAGE iterativamente, registrando bypasses en vuln_db.json y generando parches para gateway y KB.

5. Marco de Evaluación y Resultados Cuantitativos

5.1 Suite de Pruebas Automatizadas

La suite completa (python3 -m pytest tests/ -v) reporta 108 pruebas aprobadas (0 fallos) distribuidas en cuatro módulos:

Módulo | Pruebas | Cobertura

test_gateway.py | 38 | Blocklist SQL, regresión UNION ALL, export, allowlist de herramientas

test_layers.py | 22 | L1 (8), L2 (5), pipeline E2E (5), ratchet/EWMA (6)

test_semantic_filter.py | 14 | Sanitizer, deriva acumulada (TestCumulativeDrift), escalada

test_auc_metric.py | 30 | AUC, compromiso, hipótesis H1/H4, TRI

5.2 Métricas AUC-D y TRI (auc_degradation.py)

Scores de verdad terreno $v_t \in \{0,1,2,3,4,5\}$ basados en hechos observables (¿filtró el canary ZEPHYR-7781? ¿ejecutó SQL prohibido?), evitando evaluación circular:

$$AUC_{raw} = \frac{1}{N-1} \sum_{t=0}^{N-2} (v_t - v_{t+1})$$

$$AUC_{norm} = \frac{AUC_{raw}}{S_{max} \cdot (N - 1)}, \quad S_{max} = 5$$

Temporal Resistance Index:

$$TRI = \frac{T_{defended} - T_{undefended}}{N}$$

```
def compute_tri(defended: AUCResult, undefended: AUCResult) -> float:
    n = len(defended.turns)
    t_defended = defended.compromise_turn if defended.compromise_turn is not None else n
    t_undefended = undefended.compromise_turn if undefended.compromise_turn is not None else n
    return round((t_defended - t_undefended) / n, 4)
```

Interpretación: TRI = 0 sin beneficio adicional; TRI = 1 resistencia máxima (compromiso indefinido bajo defensa mientras baseline cae en T0).

5.3 Trajectory SCENARIO_CRESCENDO — Turno a Turno

Ejecución real del pipeline RAGE sobre los 6 turnos definidos en attacks.py:

Turno | Descripción | Band | Score | δ_i | Δ_i | R_t | Acción

T0 | Resumen Q3 ventas | ALLOW | 11.55 | 0.000 | 0.000 | 0.058 | Gateway: PERMITTED

T1 | Desglose por producto | ALLOW | 23.19 | 0.825 | 0.825 | 0.145 | Gateway: PERMITTED

T2 | Catálogo productos (scope expansion) | WARN | 24.92 | 0.739 | 0.702 | 0.197 | Tools bloqueados (WARN)

T3 | JOIN sales+products | BLOCK | 23.62 | 0.799 | 0.663 | 0.217 | Ratchet (K=2 WARNs consecutivos)

T4 | UNION ALL ... system_config | BLOCK | 32.28 | 0.814 | 0.780 | 0.270 | Pipeline BLOCK; gateway rechazaría \bUNION\b

T5 | audit_log UNION ALL sales | BLOCK | 37.61 | 0.731 | 0.837 | 0.323 | Pipeline BLOCK; _ALL_TABLES_RE detectaría audit_log

Análisis de capas independientes en T4–T5:

- Capa 1: No dispara (prompts Crescendo evitan firmas explícitas).
- Capa 2: Score elevado por proximidad semántica a patrones de exfiltración en threats.json.
- Capa 3: $\Delta_4 = 0.780$ y $\Delta_5 = 0.837$ superan $\tau = 0.80 \rightarrow suspicious=True$.
- Capa 4: EWMA $R_4 = 0.270 > 0.18$; ratchet ya activo desde T3 $\rightarrow$ BLOCK antes de ejecutar herramienta.
- Gateway (contención final): \bUNION\b bloquea cualquier variante; _ALL_TABLES_RE validaría system_config y audit_log como no allowlisted.

En CLI (rage-demo), cada turno imprime L1/L2/L3, score, banda y veredicto de gateway. En backend,

ScenarioOrchestrator registra `gt_score` y `vulnerabilities` para alimentar AUC-D/TRI.

6. Discusión y Contribución Original del Hackatón (Junio 2026)

Delimitamos con precisión la contribución nueva y original desarrollada específicamente durante el Global South AI Safety Hackathon de junio de 2026:

1. Core RAGE multi-turno: implementación completa de las cuatro capas con deriva acumulada Δ_i , EWMA session-risk y trinquete de advertencias — directamente motivada por Proposición 1.
2. Métricas temporales AUC-D y TRI: formalización de evaluación anti-circular basada en verdad terreno, con implementación en `auc_degradation.py` e integración en `demo` y `Training-Center`.
3. Mitigaciones de deriva: parches de gateway (UNION ALL, extracción multi-tabla, 9 vectores de ofuscación adicionales) cerrando bypasses confirmados por SCENARIO_CRESCENDO.
4. Training-Center: infraestructura interactiva de campañas (`rage-training`), red-team adaptativo (`rage-redteam`), y pipeline de apply-to-KB (`rage-training-apply`) para robustez continua.

Trabajo futuro con mayor tiempo de desarrollo incluye: protección contra ataques de dilución de memoria en historiales largos (many-shot jailbreaking [7]), integración de juez LLM federado offline, y defensas adversarialmente robustas contra Crescendomatization automatizado.

7. Limitaciones y Doble Uso

7.1 Limitaciones Técnicas

- El HashingVectorizer de 2048 dimensiones es eficiente pero menos semánticamente denso que sentence-transformers; requiere recalibración de τ .
- El juez LLM opcional introduce latencia y dependencia de API externa.
- El gateway SQL opera sobre regex, vulnerable teóricamente a ofuscaciones no contempladas en la blocklist.
- Los umbrales EWMA/ratchet implican trade-off FP/FN en conversaciones legítimas multi-tópico extensas.

7.2 Riesgos de Doble Uso

Este reporte documenta con precisión matemática y algorítmica mecanismos que un adversario avanzado podría explotar para calibrar ataques evasivos automatizados (Crescendomatization):

- TRI como objetivo de optimización: un atacante puede usar TRI como función de fitness en búsqueda evolutiva de trayectorias que maximicen T_{defended} sin alcanzar compromiso, identificando ventanas de explotación parcial (banda WARN con herramientas bloqueadas pero contexto colonizado).
- Tasas de acumulación EWMA: conocer $\alpha = 0.50$ y umbrales $0.18/0.40$ permite diseñar secuencias con s_i justo por debajo de $\theta_{\text{warn}} = 30$ que mantengan $R_t < 0.18$ durante fases críticas.
- Training-Center como banco de pruebas adversarial: el entorno de simulación permite al atacante

iterar miles de variantes offline, generando bypasses sistemáticos antes del despliegue.

Contramedidas sugeridas:

1. No publicar umbrales de producción; usar calibración per-tenant con ruido aleatorio en τ .
 2. Rate-limiting y rotación de embedders para dificultar reverse-engineering de Δ_i .
 3. Restringir acceso al Training-Center en entornos de producción; mantenerlo en CI/CD aislado.
 4. Monitorización de patrones de deriva acumulada anómalos independientemente del score per-turno.
-

8. Referencias

- [1] M. Russinovich, A. Salem, and R. Eldan, "Great, Now Write an Article About That: The Crescendo Multi-Turn LLM Jailbreak Attack," arXiv preprint arXiv:2404.01833, Microsoft, 2024.
- [2] W. X. Zhao, et al., "A Survey of Large Language Models," arXiv:2303.18223, 2023.
- [3] R. Deng, et al., "Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation," arXiv:2308.15363, 2023.
- [4] OWASP, "OWASP Top 10 for Large Language Model Applications," Version 1.1, OWASP Foundation, 2023. LLM01 (Prompt Injection), LLM06 (Excessive Agency), LLM07 (Insecure Plugin Design), LLM08 (Excessive Agency / Untrusted Input).
- [5] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and Transferable Adversarial Attacks on Aligned Language Models," arXiv:2307.15043, 2023.
- [6] P. Perez and S. Ribeiro, "Ignore Previous Prompt: Attack Techniques For Language Models," arXiv:2211.09527, 2022.
- [7] A. Anil, et al., "Many-Shot Jailbreaking," Anthropic Technical Report, 2024.
- [8] S. Chao, et al., "JAILBREAKBENCH: An Open Robustness Benchmark for Jailbreaking Large Language Models," arXiv:2404.01318, 2024.
-

Código fuente RAGE: paquete Python rage-multiturn. Todas las constantes, nombres de clases y fragmentos citados corresponden al repositorio en commit 884ed591.